

# Git Explainer Agent Design Document

## Git Explainer Agent — Design Document

CIS 1990 Final Project. Authors: Andrei and Alistair.

To understand how to use the project, see README.md. Eval results and critique are in eval/testing\_metrics.md.

### 1. Problem, user, motivation, and scope

**Problem.** Developers often inherit code whose history is spread across line-level git metadata, commit messages, pull requests, issue threads, review comments, and surrounding source context. Looking this up manually is slow and error-prone, especially when a line has been renamed, moved, refactored, or touched by several commits.

**Intended user.** The primary user is a developer, reviewer, teaching assistant, or maintainer working in a local clone who wants to answer "why does this code exist?" without manually chasing `git blame`, `git log`, GitHub PRs, and issue links. The user is assumed to be able to run a CLI and inspect JSON, but not necessarily to know the full project history.

**Motivation.** The project is designed to make code history explanations more auditable than a normal chatbot answer. Instead of only producing prose, it returns the retrieved commits, pull requests, issues, file snippets, and diffs that support the explanation, and it requires citations in synthesized claims so users can check the answer.

**Project scope.** The agent takes a local git clone plus either a line range or a natural-language question, and returns a structured JSON ExplanationResult with five synthesis sections (`what_changed`, `why`, `tradeoffs`, `limitations`, `summary`) plus the commits, pull requests, issues, file contexts, and diffs that back them, cache statistics, and fallback / condensation audit fields.

Input shapes, validated in guardrails.py:41:

1. (`repo_path`, `file_path`, `start_line`, `end_line`) — direct line-range mode.

2. (`repo_path`, `--question ...`) with an optional `file_path` hint — resolved to a concrete span before tracing.

Out of scope: non-GitHub hosting, private repos (refused by default, `enforce_public_repo=True`, opt out via `--allow-private-repo`), binary files, cross-repository traces, and fully natural-language repository-wide search that is not tied back to a concrete file span.

## 2. System flow

A single invocation of `python main.py ...` follows this condensed pipeline:

- **Entry mode:** the CLI accepts either direct line-range input (`repo file_path start end`) or question mode (`repo --question ...` with an optional file hint).
- **Validation:** `validate_query` checks repository scope, file safety, line ranges, public-repo policy, and bounded parameters before expensive work begins. Invalid input raises `ValueError`.
- **Question resolution:** question mode uses `resolve_question_to_code` to map the question to a concrete file span, then re-runs validation on that resolved span.
- **History tracing:** `trace_line_history` finds commits that shaped the selected lines. If no commits are found, `search_commits` provides a last-resort `git log` fallback.
- **Evidence collection:** for each commit up to `max_commits`, the orchestrator gathers PR metadata, PR comments, linked issues, issue comments, file context, and redacted diffs.
- **Memory layer:** `ExplainerMemory` checks cached PRs, issues, comments, file contexts, and diffs before fetching; every cache miss is written back and flushed at the end.
- **Context control:** `condense_evidence` compresses long PR or issue text only when the evidence payload exceeds `EVIDENCE_CHAR_BUDGET`.
- **Synthesis:** if `use_llm` is enabled and the provider is available, `llm.chat` synthesizes the cited explanation; otherwise `_fallback_summary` builds deterministic prose.
- **Citation enforcement:** `_ensure_citation_coverage` rejects uncited synthesized sentences, retries once, then falls back deterministically if coverage still fails.
- **Output:** the CLI prints an `ExplanationResult` JSON object. The returned evidence remains uncondensed; only the synthesis prompt sees the condensed copy.

### 3. Design rationale by required component

#### Data incorporation

The agent incorporates four main evidence sources: local git history, GitHub pull request metadata, GitHub issue metadata, and source-level context from diffs plus surrounding file snapshots. Local git history is the grounding layer because the user asks about a concrete file span or a question that is resolved into one. GitHub PRs, review comments, issues, and issue comments add intent signals that are often absent from commit messages.

This design prioritizes auditability over broadness. Every retrieved artifact is returned in the JSON output so the explanation can be checked against the evidence. The main alternative would be a more open-ended web or repository search, but that would make it harder to prove why a particular piece of evidence was relevant. The limitation is that rationale not captured in commits, PRs, issues, review comments, or nearby code will not be visible to the agent.

#### Memory and retrieval

Retrieval starts with line history because the line span is the most specific signal the user provides. In question mode, the system first uses `question_resolver` to map natural-language text to a concrete file span, then runs the same history pipeline. `ExplainerMemory` caches GitHub and context lookups in `.git_explainer_cache.json` so repeated runs avoid unnecessary API calls and keep evaluation runs more stable.

The cache is intentionally simple and local. A database or vector index would support richer semantic retrieval, but would add setup burden and make the evidence path less transparent. The tradeoff is that the JSON cache can become stale if GitHub conversations change, and it is not designed for concurrent writes by many processes.

#### Tools

The system uses small deterministic tools rather than letting an LLM freely choose arbitrary shell commands. Three central examples are `git_blame_trace`, `github_pr_lookup`, and `git_diff_reader`. `git_blame_trace` answers which commits shaped the selected lines; `github_pr_lookup` enriches those commits with review and PR context; `git_diff_reader` extracts compact, redacted change summaries for the synthesis step.

The main reason for this tool design is control. Narrow tools are easier to test, cite, cache, and threat-model than a general command executor. The tradeoff is less flexibility: the fixed pipeline may miss unusual evidence that a human investigator would search for manually. The optional planner/critic path can add more adaptive behavior, but the default path keeps retrieval deterministic.

Implemented tools. Each tool under `git_explainer/tools/` is a thin module with one job:

- **git\_blame\_trace** — primary tracer. `git log -L` first, then `git blame -M` with `.git-blame-ignore-revs` and `git log --follow -M` as fallbacks.
- **github\_pr\_lookup** — `find_prs_for_commit`, `fetch_pr`, `fetch_pr_comments`.
- **github\_issue\_lookup** — `extract_issue_refs`, `fetch_issue`, `fetch_issue_comments`.
- **file\_context\_reader** — reads file contents at a given revision.
- **git\_diff\_reader** — compact per-commit diff summaries with credential redaction (`_redact_sensitive_diff_content`).
- **commit\_search** — last-resort `git log` wrapper used when line tracing returns nothing.
- **question\_resolver** — maps a natural-language question to a concrete line span using AST parsing for Python files and keyword matching elsewhere. Not an LLM call.

All external fetches are cached in the JSON-backed `ExplainerMemory` (stored at `.git_explainer_cache.json` inside the target repo, seven buckets keyed by shape).

## Robust system design

The robust path has several layers: input validation before any costly work, fallback commit search when line tracing returns no commits, evidence condensation when prompt payloads are too large, LLM synthesis only when available, citation validation after synthesis, and a deterministic fallback summary when the LLM is disabled or fails.

This design assumes the agent should usually return a limited, inspectable answer instead of failing just because the LLM is unavailable. It also assumes that unsupported fluent prose is worse than a plain fallback. The limitation is that deterministic fallback summaries are less nuanced, and citation coverage checks only verify that claims have citation-shaped support, not that every cited claim is semantically proven.

## Guardrails

The guardrails constrain both the user-facing input and the evidence that reaches the model. They validate line ranges, reject missing or binary files, enforce repository containment, cap request sizes, refuse private repositories by default, redact likely credentials from diffs, and reject synthesized prose that lacks citations.

The design goal is to keep the agent useful for normal code-history questions while reducing risk from path traversal, prompt injection, credential exposure, private repository leakage, and hallucinated answers. The main tradeoff is con-

servative behavior: some legitimate private or offline workflows require an explicit opt-out (`--allow-private-repo`), and some useful large queries must be narrowed to stay within span and context limits.

Below is a list of guardrails:

- **Line span** capped at `DEFAULT_MAX_LINE_SPAN = 200` (guardrails.py:71-76).
- **Positive integers and ordering:** `start_line`, `end_line`  $> 0$  and `end_line`  $\geq$  `start_line` (guardrails.py:66-69).
- **File existence:** missing or binary files raise (guardrails.py:117-125).
- **Repository containment:** `normalize_file_path` rejects paths outside the repo root.
- **Private-repo refusal (default on):** `enforce_public_repo` now defaults to `True`. The guardrail calls `ensure_public_github_repo`, which rejects 404 or `private: true`. Opt out at the CLI with `--allow-private-repo` or programmatically by constructing `Explainer-Query` with `enforce_public_repo=False`.
- **Parameter clamping:** `max_commits` stays in  $[1, 20]$ , and `context_radius` stays in  $[0, 200]$  (guardrails.py:102-103).
- **Citation coverage:** `_ensure_citation_coverage` rejects synthesized sentences without a bracketed citation and triggers the retry loop.

## Evaluation

Evaluation is built around benchmark cases rather than only manual inspection. The harness checks whether the agent retrieves expected commits, PRs, and issues; whether explanations include citations; whether citations resolve to returned evidence; whether invalid inputs fail safely; and whether latency remains reasonable. The evaluation also distinguishes fallback-only runs from LLM-enabled runs.

This design separates retrieval correctness from explanation quality. That matters because an answer can retrieve the right commits while still summarizing them poorly, or produce well-formatted citations that do not fully support the prose. The limitation is that the deterministic faithfulness score is a proxy rather than a human judgment, and LLM judge results, when used, are still model-based rather than definitive.

## 5. Evidence pre-summarization (condensation)

Long GitHub threads can easily overflow the synthesis model's context window. Between evidence collection and synthesis, the orchestrator invokes `condense_evidence` on the collected payload (commits, pull requests, issues, file contexts, diffs).

Trigger threshold. If the serialized evidence dict is at or under `config.EVIDENCE_CHAR_BUDGET` (default 30000 characters, overridable

via the `EVIDENCE_CHAR_BUDGET` env var), condensation is a no-op and the report's `method_used` is `"none"`. Only when the payload exceeds the budget does the condenser run.

Two-tier strategy. For each eligible field, longest first:

1. Tier 1 (preferred): the LLM is asked for a concise summary (`EVIDENCE_SUMMARY_TARGET_CHARS`, default 800) that explicitly preserves commit SHAs, PR/issue numbers, file paths, technical trade-offs, and stated intent. Output is prefixed with `[pre-summarized]` in the condensed copy.
2. Tier 2 (fallback): deterministic head+tail truncation with a visible elision marker (`[... content truncated: N chars elided ...]`). Used when the LLM is unavailable or returns an empty reply. Output is prefixed with `[truncated]`.

Fields touched vs. preserved. Condensation is intentionally narrow:

- **Condensed:** `pull_requests[i].body`, `pull_requests[i].review_comments[j].body`, `issues[i].body`, `issues[i].comments[j].body`, only when length exceeds `config.EVIDENCE_FIELD_MAX_CHARS` (default 3000).
- **Preserved verbatim:** all commit SHAs (full and short), PR/issue numbers, titles, labels, URLs, `file_contexts` entries, `diffs` entries, and any other structural metadata.

Report shape. The condenser returns a `CondensationReport` serialized as the `condensation` field of the `ExplanationResult`:

```
"condensation": {
  "original_size": 48123,
  "condensed_size": 22041,
  "fields_condensed": ["pr#42.body", "issue#7.comments[2].body"],
  "method_used": "llm"    // "none" | "llm" | "heuristic" | "mixed"
}
```

Caller visibility. The `ExplanationResult` returned to the caller still contains the **un-condensed originals** for `pull_requests`, `issues`, `file_contexts`, and `diffs`. Only the synthesis LLM sees the condensed view, via `build_synthesis_prompt(condensed_evidence, ...)` in `orchestrator.py`. Downstream consumers (notebooks, eval harness, `--use-llm-judge`) therefore score the agent against the full evidence, not the compressed view.

## 6. Threat model

The main risks and controls are:

- **Unsafe line-range input:** invalid spans, non-positive line numbers, oversized ranges, missing files, binary files, and paths outside the repo are blocked by `validate_query` and `normalize_file_path`.

- **Unsafe question input:** empty, punctuation-only, or stopword-only questions are rejected by `resolve_question_to_code`, which raises when no specific search terms remain.
- **Prompt injection in user questions:** question resolution is deterministic keyword and path scoring, not an LLM call, so instruction-shaped user text is treated as data.
- **Prompt injection in fetched evidence:** `SYSTEM_PROMPT` grounds claims in returned evidence; `_ensure_citation_coverage` rejects uncited prose; `_fallback_summary` avoids free-form model text when needed.
- **Unsafe tool use:** subprocess calls use `argv` lists with `shell=False`, GitHub fetches are scoped to `api.github.com`, and file reads are scoped to the resolved repository root.
- **Privacy leakage:** `ensure_public_github_repo` refuses private repositories by default; `GITHUB_TOKEN` is only sent to GitHub API requests and is not logged.
- **Sensitive data in diffs:** `_redact_sensitive_diff_content` masks likely tokens, passwords, API keys, auth headers, and URL credentials before diffs enter the evidence payload.
- **Hallucinated model output:** citation retry runs up to two attempts, then the orchestrator returns the deterministic fallback instead of unsupported prose.
- **Rate-limit and API failures:** GitHub helpers raise clearly on 401, 403, and 429, and retry transient failures with exponential backoff.

## 7. Evaluation

Scored by `eval/evaluate.py` against 20 benchmark cases in `eval/benchmark.json`. `eval/results.json` is the fallback-only run covering 16 cases (the other four require `use_llm=True` or an external repo clone).

Evaluation methodology. The benchmark suite is organized around task types that correspond to the agent's main responsibilities:

- **Line-range explanation tasks:** the user supplies a repository, file, and line span. Success means the agent validates the input, retrieves the expected commits and related PR/issue evidence, and produces a cited explanation for the selected code.
- **Question-resolution tasks:** the user supplies a natural-language question, optionally with a file hint. Success means the resolver maps the question to the expected file/span or matched terms before the normal history-tracing pipeline runs.
- **Thin-evidence and ambiguous-history tasks:** the selected code is connected to multiple commits, weak PR descriptions, or no linked issues. Success means the answer surfaces uncertainty and limitations rather than inventing a single unsupported rationale.
- **LLM/fallback behavior tasks:** some cases run with `use_llm=True` while others force `--no-llm`. Success means the agent either returns an

LLM synthesis that passes citation checks or falls back to the deterministic summary with the correct `used_fallback` behavior.

- **Failure cases:** invalid repositories, missing files, binary files, impossible line ranges, and end-line values beyond file length should fail clearly with validation errors instead of running partial tool calls or returning misleading explanations.
- **Adversarial cases:** prompt-injection-shaped questions, path traversal attempts, and cases that should not return PR/issue metadata test whether the system treats user text as data, keeps file access inside the repo, and avoids unsupported evidence claims.

The primary success criteria are retrieval recall against hand-authored gold commits/PRs/issues, citation coverage, citation validity, expected fallback behavior, correct question resolution, and correct abstention when PR or issue evidence should be absent. Latency is tracked as a secondary operational metric. Failure and adversarial cases are judged by refusal quality, absence of unsafe tool behavior, and whether the agent avoids fabricating evidence. The faithfulness score is treated as a proxy because it is deterministic and not a human judgment.

| Metric               | Target | Actual                                            |
|----------------------|--------|---------------------------------------------------|
| Retrieval accuracy   | 85%    | 100% (5 / 5 gold targets, notebook run)           |
| Summary faithfulness | 80%    | 5.00 / 5.00 on proxy rubric (not human-rated)     |
| Citation coverage    | 100%   | 100% (27 / 27 citable sentences)                  |
| Citation validity    | —      | 100% (53 / 53 citations resolve to real evidence) |
| Pass rate            | —      | 16 / 16 on the non-LLM subset                     |
| Latency p50          | —      | 0.063s fallback / 0.150s LLM                      |
| Latency p95          | —      | 1.094s fallback / 0.192s LLM                      |

## 8. User transcripts

### Transcript 1: Successful line-range query with retrieved evidence

User command.

```
python main.py . git_explainer/guardrails.py 41 60 --no-llm --owner AndreiPiterbarg --repo-r
```

System response excerpt.

```
{
  "commits": [
    {"sha": "b05641e", "message": "adjust to handle natural language query"},
    {"sha": "3870c34", "message": "initial mockup"}
  ],
  "pull_requests": [
    {"number": 1, "title": "initial mockup", "state": "merged"}
  ],
  "explanation": {
```

```

    "what_changed": "The selected lines in git_explainer/guardrails.py:41-60 were most recent",
    "why": "Related pull requests suggest the intent was #1 (initial mockup). [pr:#1]",
    "limitations": "This explanation is limited to the traced commits, associated pull requests",
  },
  "used_fallback": true
}

```

**Behavior shown.** This is a clean-success path. The agent validates the line range, traces two relevant commits, fetches PR metadata, builds file context and diff evidence, and returns a cited five-section explanation. Because the run uses `--no-llm`, `used_fallback: true` confirms that the deterministic fallback summary produced the final prose.

## Transcript 2: Difficult case with multi-commit, ambiguous history

User command.

```
python main.py . git_explainer/config.py 13 19 --no-llm --owner AndreiPiterbarg --repo-name C2
```

System response excerpt.

```

{
  "commits": [
    {"sha": "3870c34", "author": "aking526", "message": "initial mockup"},
    {"sha": "4c711bf", "author": "AndreiPiterbarg", "message": "Switch LLM provider from Kin"},
    {"sha": "628ef3c", "author": "AndreiPiterbarg", "message": "Add project configuration an"},
  ],
  "explanation": {
    "what_changed": "The selected lines in git_explainer/config.py:13-19 were most recently",
    "why": "Related pull requests suggest the intent was #1 (initial mockup) [pr:#1].",
    "limitations": "This explanation is limited to the traced commits, associated pull requests",
  },
  "used_fallback": true
}

```

**Behavior shown.** The chosen config lines were touched by multiple commits from different authors: initial configuration, provider migration, and later refactoring. The system does not collapse that evidence into a single unsupported narrative. It cites all three commits, includes the related PR when available, and uses the limitations field to flag that intent may be incomplete when discussion is absent from retrieved metadata.

## Transcript 3: Safety case — adversarial low-signal question rejected by C2

User command.

```
python main.py . --question "explain how this is used" --owner AndreiPiterbarg --repo-name C2
```

System response (stderr, exit code 1).

```

Traceback (most recent call last):
  File "main.py", line 107, in <module>
    main()
  File "main.py", line 88, in main
    result = explain_code_history(
  File "git_explainer/orchestrator.py", line 876, in explain_code_history
    return agent.explain(query)
  File "git_explainer/orchestrator.py", line 110, in explain
    resolution = resolve_question_to_code(
  File "git_explainer/tools/question_resolver.py", line 139, in resolve_question_to_code
    raise ValueError("question must include at least one specific search term")
ValueError: question must include at least one specific search term

```

**Behavior shown.** The question is an imperative-styled probe ("explain how this is used") whose every token — `explain`, `how`, `this`, `is`, `used` — is in the resolver's stopword set (`question_resolver.py:38-85`). After stopword filtering the term list is empty, so `_extract_question_features` returns no `terms`, and `resolve_question_to_code` raises at line 139 — control **C2** in the threat model. The agent never reaches `trace_line_history`, never calls the GitHub API, and never invokes the synthesis LLM, so a vacuous adversarial prompt cannot trick the model into fabricating an answer or enumerating repository contents. The same guard fires for empty questions and pure-punctuation questions because both reduce to an empty term list.